Devixo Solutions
Summer Internship Program

Task 03
End-to-End Machine Learning Project

Customer Churn Prediction

Student Name	Amtul Jamil Ansa
Internship Domain	AI / ML Internship
Task Number	Task 03 (Week 3)
Dataset	Telco Customer Churn (Kaggle)
Submission Date	November 2026

1. Objective

The objective of this task is to develop a complete, end-to-end machine learning project, from data collection through to model evaluation. The project applies feature engineering, model optimization and a reusable pipeline to a real-world dataset, while following clean, maintainable coding practice.

2. Problem Statement

Telecom companies lose revenue every time a customer cancels their service, which is known as customer churn. Keeping an existing customer is usually cheaper than winning a new one, so being able to identify customers who are likely to leave, before they do, lets the company offer targeted retention incentives.

This project builds a binary classification model that predicts whether a customer will churn (Yes or No) from the customer's demographics, the services they subscribe to, and their account and billing information.

3. Technologies Used

Technology	Purpose in this project
Python 3	Programming language for the whole project
Google Colab	Cloud notebook environment used to run and document the code
Pandas, NumPy	Loading, cleaning and transforming the data
Matplotlib, Seaborn	Exploratory plots and result charts
Scikit-learn	Pipelines, scaling, feature selection, models, tuning, cross validation and metrics
XGBoost	Optional gradient boosting model
Joblib, Pickle	Saving the best model to disk (Part 4)
Streamlit	Bonus: web interface for making predictions

4. Dataset

The project uses the **Telco Customer Churn** dataset from Kaggle (blastchar/telco-customer-churn). It contains **7,043 customers and 21 columns**, which satisfies the minimum requirement of 3,000 rows. Each row describes one customer. The target column, **Churn**, records whether the customer left the company.

Column group	Columns
Identifier	customerID
Demographics	gender, SeniorCitizen, Partner, Dependents
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Account and billing	tenure, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
Target	Churn (Yes / No)

5. Implementation Steps

The work follows the five parts of the task description. Each step below is explained with screenshots of the code and its output from the Google Colab notebook.

1. **Part 1 - Data preparation:** import the data, handle missing values, remove duplicates, engineer features, select features and scale the data.
2. **Part 2 - Model development:** train Decision Tree, Random Forest, Support Vector Machine, Gradient Boosting and XGBoost models.
3. **Part 3 - Model optimization:** hyperparameter tuning, cross validation and feature importance analysis.
4. **Part 4 - Save the best model:** store it with Joblib and Pickle.
5. **Part 5 - Documentation:** dataset, workflow, model selection, results, limitations and future improvements.
6. **Bonus:** a Streamlit prediction interface.

6. Part 1 - Data Preparation

6.1 Environment Setup

0. Setup

We import the libraries and fix a random seed so results are reproducible. `N_ITER` controls how many rar combinations are tried per model (higher = better search but slower).

[2] ✓ 4s

```
!pip install -q kagglehub

import os, glob, json, time, pickle, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from scipy.stats import randint, uniform, loguniform

from sklearn.base import clone
from sklearn.model_selection import (train_test_split, StratifiedKFold, GridSearchCV,
                                     RandomizedSearchCV, cross_val_score, cross_validate)
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import mutual_info_classif
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
```

Figure 1: Setup cell: installing kagglehub and importing the libraries

[2] ✓ 4s

```
from sklearn.inspection import permutation_importance

try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except ImportError:
    HAS_XGB = False

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

RANDOM_STATE = 42
N_ITER = 15        # random-search iterations per model
CV_FOLDS = 5
print("Setup complete. XGBoost available:", HAS_XGB)
```

```
Setup complete. XGBoost available: True
```

Reusable helper module

To make the project *reusable*, all data-cleaning and feature-engineering logic lives in one function, `prepare()`, saved to a file called `churn_utils.py`. The same function is used for training here **and** for predictions in the Streamlit app, so raw customer data is always processed exactly the same way.

[3] ✓ 0s

```
%%writefile churn_utils.py
```

Figure 2: Remaining imports, random seed and configuration values

The first cell installs **kagglehub**, which is used to download the dataset from Kaggle, and imports every library the project needs: pandas and NumPy for data handling, Matplotlib and Seaborn for

plots, and scikit-learn for preprocessing, models, hyperparameter search and metrics. XGBoost is imported inside a try/except block so the notebook still runs if it is not installed.

A fixed random seed (**RANDOM_STATE = 42**) makes every result reproducible. **N_ITER = 15** sets how many random hyperparameter combinations are tried per model in Part 3, and **CV_FOLDS = 5** sets the number of cross-validation folds. The output "Setup complete. XGBoost available: True" confirms that the environment is ready.

6.2 Reusable Helper Module (churn_utils.py)

[3] ✓ 0s

```
import numpy as np
import pandas as pd

YES_NO_COLS = ["Partner", "Dependents", "PhoneService", "PaperlessBilling"]
SERVICE_COLS = ["MultipleLines", "OnlineSecurity", "OnlineBackup", "DeviceProtection",
                "TechSupport", "StreamingTV", "StreamingMovies"]
CATEGORICAL_COLS = ["InternetService", "Contract", "PaymentMethod", "tenure_group"]


def prepare(df, feature_columns=None):
    """Clean raw Telco customer data, engineer features and encode it numerically.

    If `feature_columns` is given, the output is re-indexed to exactly those columns
    (missing dummy columns are filled with 0) so a single customer can be scored.
    """
    d = df.copy()
    d = d.drop(columns=[c for c in ["customerID", "Churn"] if c in d.columns])

    # TotalCharges is stored as text; blanks belong to brand-new customers (tenure = 0)
    d["TotalCharges"] = pd.to_numeric(d["TotalCharges"].astype(str).str.strip(),
                                      errors="coerce").fillna(0)

    # "No internet service" / "No phone service" carry the same meaning as "No"
    for c in SERVICE_COLS:
        d[c] = d[c].replace({"No internet service": "No", "No phone service": "No"})

    # ---------- Feature engineering ----------
    d["tenure_group"] = pd.cut(d["tenure"], bins=[-1, 12, 24, 48, np.inf],
```

Figure 3: Start of the helper module written to churn_utils.py with the %%writefile command

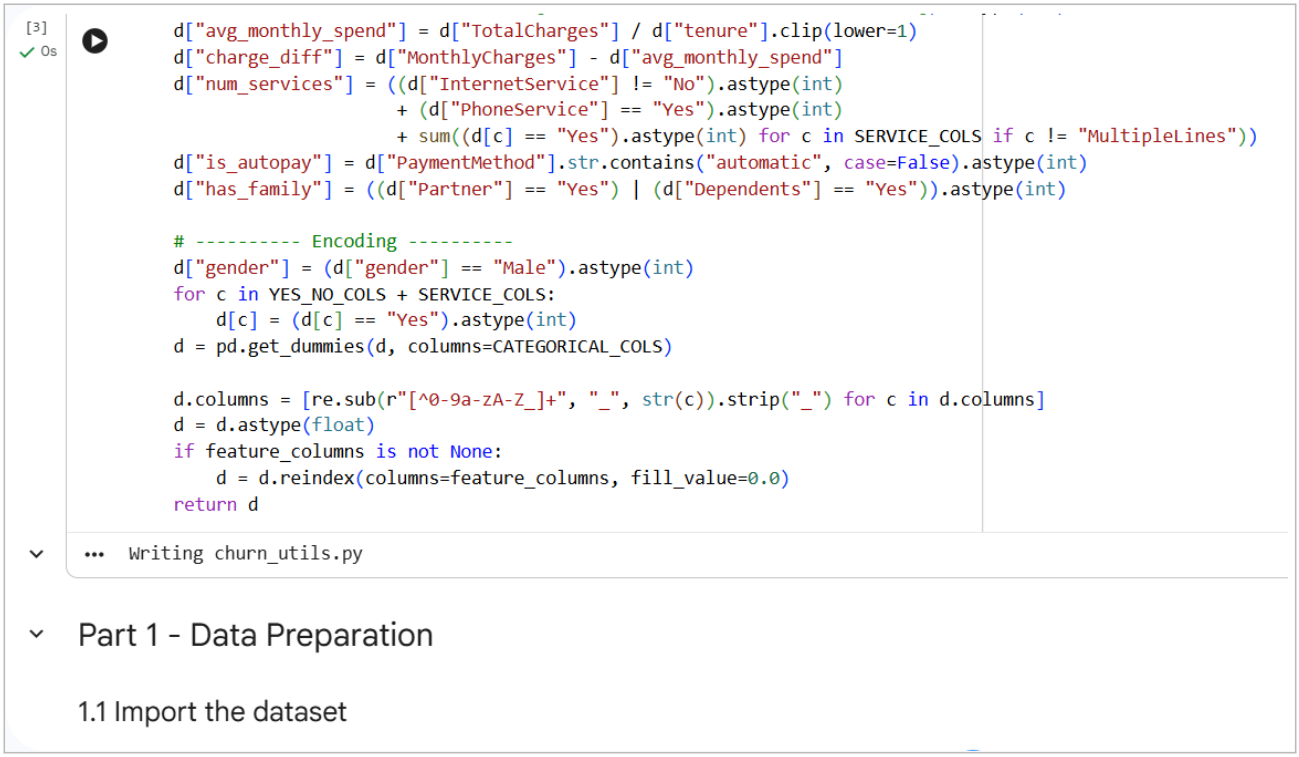


Figure 4: The prepare() function: feature engineering, encoding and column alignment

All cleaning, feature-engineering and encoding logic is written once, in a function called **prepare()**, and saved to a file named churn_utils.py. Keeping this logic in one place makes the project reusable: the same function prepares the training data in the notebook and prepares a single customer's details inside the Streamlit app, so the model always receives data in exactly the format it was trained on.

The function (1) drops customerID and Churn, (2) converts TotalCharges from text to numbers, (3) merges the values "No internet service" and "No phone service" into "No", (4) creates six engineered features, (5) converts Yes/No columns to 1/0 and one-hot encodes the remaining categorical columns, (6) cleans the column names, and (7) can re-index the result to a fixed list of columns so that a single customer can be scored.

6.3 Importing the Dataset

The notebook downloads the dataset from Kaggle automatically. If that fails, it asks you to uploa
the Kaggle link above).

[4] ✓ 4s

```
CSV_NAME = "WA_Fn-UseC_-Telco-Customer-Churn.csv"

if not os.path.exists(CSV_NAME):
    try:
        import kagglehub
        folder = kagglehub.dataset_download("blastchar/telco-customer-churn")
        csv_path = glob.glob(os.path.join(folder, "*.csv"))[0]
    except Exception as e:
        print("Automatic download failed:", e)
        from google.colab import files
        uploaded = files.upload()          # choose the Telco churn CSV
        csv_path = list(uploaded.keys())[0]
else:
    csv_path = CSV_NAME

import churn_utils                         # helper module written above
from churn_utils import prepare

df_raw = pd.read_csv(csv_path)
print("Rows, columns:", df_raw.shape)
df_raw.head()
```

Figure 5: Code that loads the dataset (automatic Kaggle download with a manual-upload fallback)

[4] ✓ 4s

```
from churn_utils import prepare

df_raw = pd.read_csv(csv_path)
print("Rows, columns:", df_raw.shape)
df_raw.head()
```

```
Using Colab cache for faster access to the 'telco-customer-churn' dataset.
Rows, columns: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No

5 rows × 21 columns

Figure 6: Dataset shape (7,043 rows, 21 columns) and the first five rows

The loading cell first checks whether the CSV file is already available. If it is not, the dataset is downloaded from Kaggle with kagglehub, and if that fails the user is asked to upload the file manually. The file is read into a pandas DataFrame. The output shows **7,043 rows and 21 columns**. The first rows show the three kinds of information held for each customer: demographics, the services they subscribe to, and account details.

```
[5]
✓ 0s
df_raw.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Figure 7: Output of df_raw.info(): column names, non-null counts and data types

df_raw.info() shows that every column has 7,043 non-null entries, so there are no obviously missing values. However, TotalCharges has the data type *object* (text) even though it should be numeric. This points to hidden formatting problems that are dealt with in the next step. Of the 21 columns, 18 are text, 2 are integers (SeniorCitizen and tenure) and 1 is a decimal number (MonthlyCharges).

```
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

[6] ✓ 0s

```
df_raw.describe(include="all").T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
customerID	7043	7043	7590-VHVEG	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612	0.0	0.0	0.0	0.0	1.0
Partner	7043	2	No	3641	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481	0.0	9.0	29.0	55.0	72.0
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN	NaN	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Figure 8: Summary statistics from df_raw.describe(include="all") (first part)

TechSupport	7043	3	No	3473	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047	18.25	35.5	70.35	89.85	118.75
TotalCharges	7043	6531		11	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN	NaN	NaN	NaN	NaN	NaN

1.2 Handle missing values

`TotalCharges` looks numeric but is stored as text, which usually hides blank values. We convert it to numbers and see w

[7] ✓ 0s

```
df = df_raw.copy()
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"].astype(str).str.strip(), errors="coerce")

missing = df.isna().sum()
print("Missing values per column (only columns with missing data):")
print(missing[missing > 0])
```

Figure 9: Summary statistics (second part)

The summary statistics describe the typical customer. Tenure ranges from 0 to 72 months with a mean of about 32.4, and MonthlyCharges ranges from 18.25 to 118.75 with a mean of about 64.76. About 16.2% of customers are senior citizens. Month-to-month is the most common contract (3,875 customers) and Fiber optic is the most common internet service (3,096 customers). In the Churn column, 5,174 of the 7,043 customers did not churn. For TotalCharges, the most frequent value is an empty entry that appears 11 times, another sign that this column needs cleaning.

6.4 Handling Missing Values and Removing Duplicates

```
[ ]
# Who are the customers with missing TotalCharges?
df[df["TotalCharges"].isna()][["customerID", "tenure", "MonthlyCharges", "TotalCharges"]].head(11)
```

```
Missing values per column (only columns with missing data):
TotalCharges    11
dtype: int64
```

	customerID	tenure	MonthlyCharges	TotalCharges
488	4472-LVYGI	0	52.55	NaN
753	3115-CZMZD	0	20.25	NaN
936	5709-LVOEQ	0	80.85	NaN
1082	4367-NUYAO	0	25.75	NaN
1340	1371-DWPAZ	0	56.05	NaN
3331	7644-OMVMY	0	19.85	NaN
3826	3213-VVOLG	0	25.35	NaN
4380	2520-SGTTA	0	20.00	NaN
5218	2923-ARZLG	0	19.70	NaN
6670	4075-WKNIU	0	73.35	NaN
6754	2775-SEFEE	0	61.90	NaN

ng preview...

Figure 10: Converting TotalCharges to numeric exposes 11 missing values, all with tenure = 0

TotalCharges was converted to a numeric type, turning invalid entries into missing values. This revealed **11 missing values**, all in TotalCharges. The listed rows show that every one of these customers has **tenure = 0**: they have only just joined and have not been billed yet. The blank therefore means "no charges so far", so filling it with **0** is more accurate than using the mean or median.

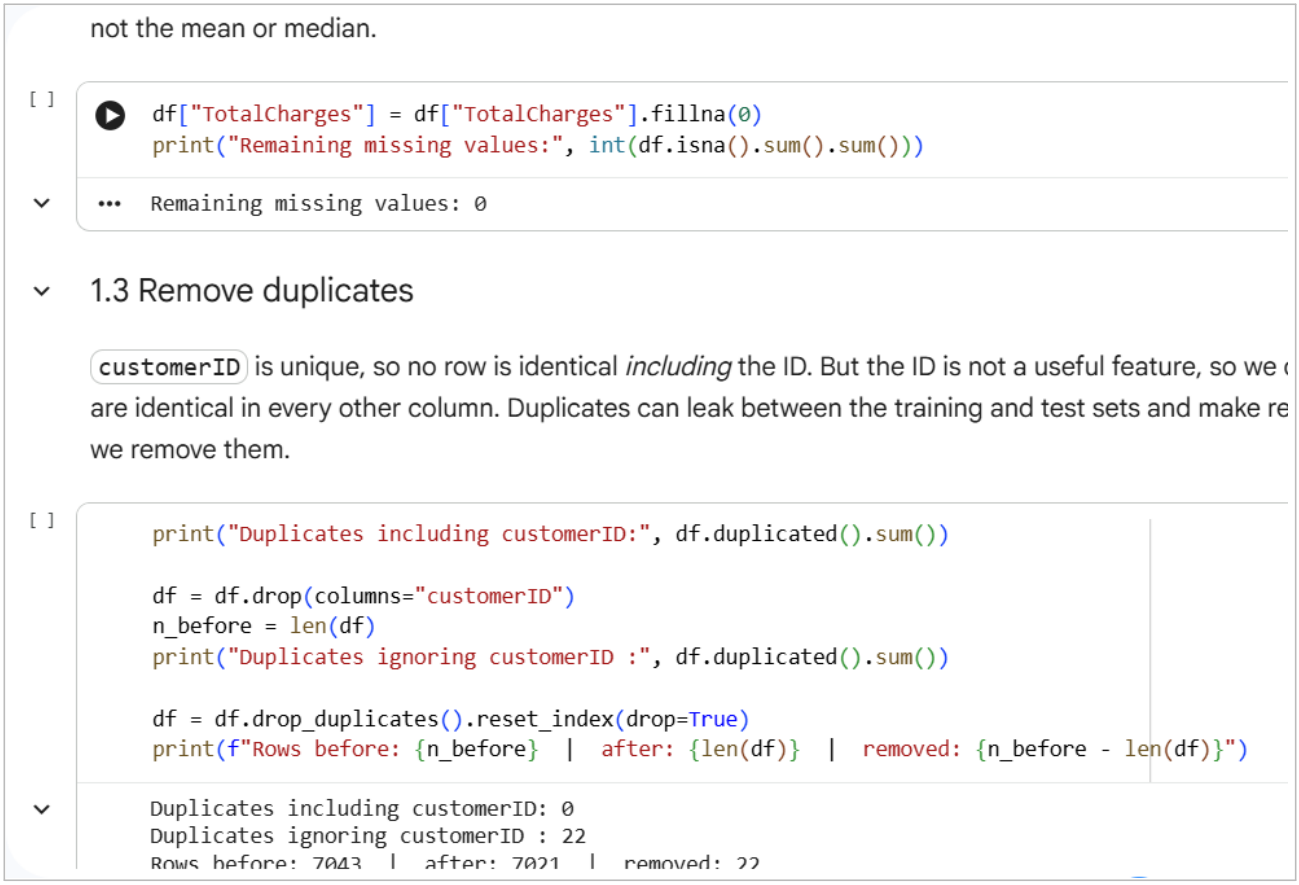


Figure 11: Filling missing values with 0 and checking for duplicate rows

After filling, the check confirms that no missing values remain. The duplicate check finds no duplicates when customerID is included, because every ID is unique. Once the ID column is dropped, however, **22 rows** are identical in every other column. These rows were removed so that the same record cannot appear in both the training and the test set, which would make results look better than they really are. The dataset now has **7,021 rows**.

6.5 Exploratory Data Analysis

1.4 Quick exploratory analysis

A few plots to understand the problem before modelling.

```
[ ]
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Target balance
counts = df["Churn"].value_counts()
axes[0, 0].bar(counts.index, counts.values, color=["#4C9F70", "#D9534F"])
axes[0, 0].set_title("Churn distribution (class balance)")
for i, v in enumerate(counts.values):
    axes[0, 0].text(i, v + 30, f"{v} ({v / len(df):.1%})", ha="center")

# Churn rate by contract
rate = df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean()).sort_values()
axes[0, 1].bar(rate.index, rate.values, color="#5B8DB8")
axes[0, 1].set_title("Churn rate by contract type")
axes[0, 1].set_ylabel("Churn rate")

# Tenure
sns.histplot(data=df, x="tenure", hue="Churn", bins=30, multiple="stack",
             palette=["#4C9F70", "#D9534F"], ax=axes[1, 0])
axes[1, 0].set_title("Tenure (months) by churn")

# Monthly charges
```

Figure 12: Code for the four exploratory plots

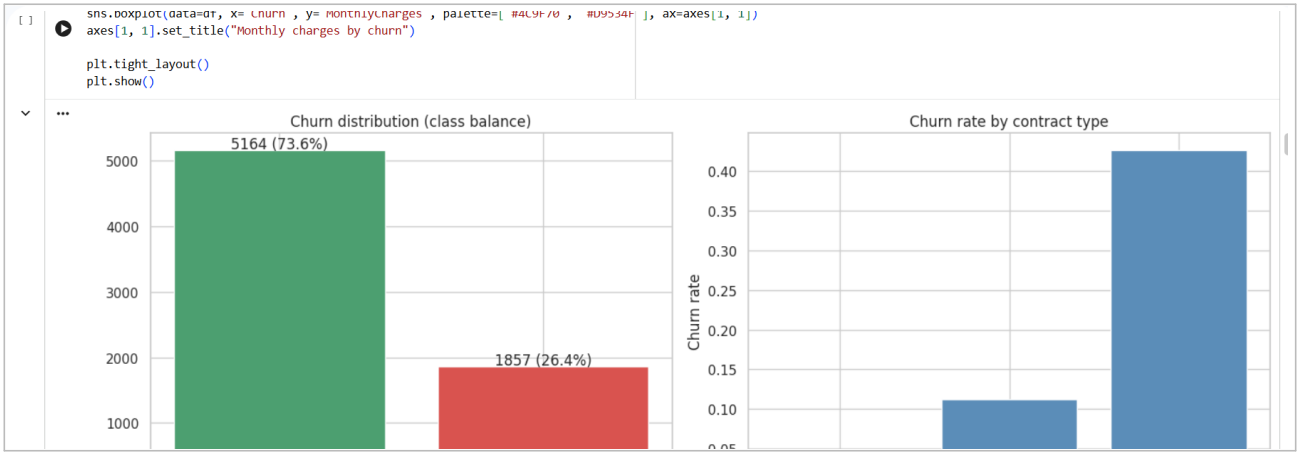


Figure 13: Plots 1 and 2: class balance and churn rate by contract type

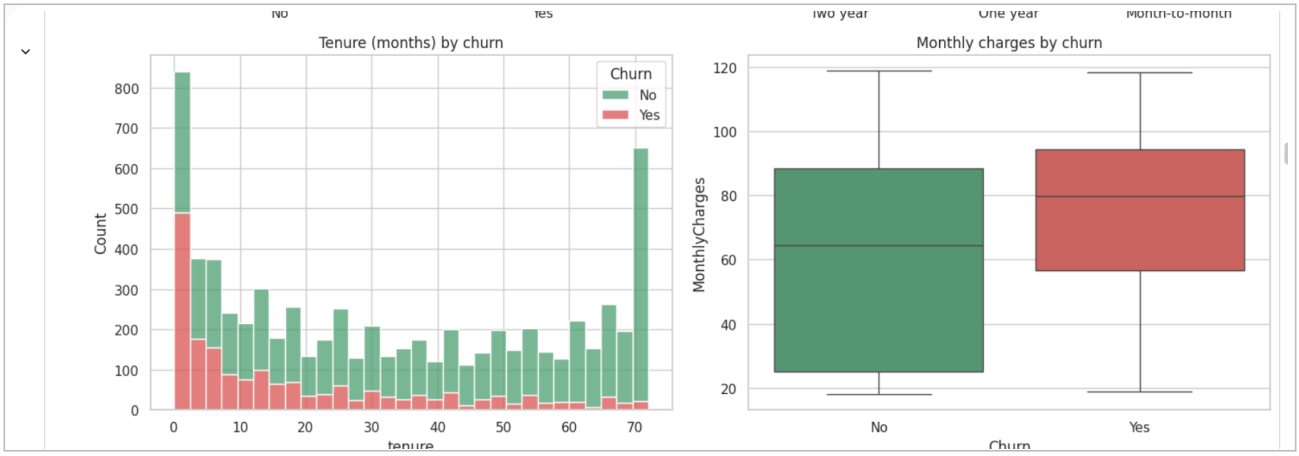


Figure 14: Plots 3 and 4: tenure by churn and monthly charges by churn

A four-panel figure was created to understand the problem before modelling.

- **Class balance:** 5,164 customers (73.6%) stayed and 1,857 (26.4%) churned. The classes are imbalanced, which is why accuracy alone is not a reliable metric and why class weights, recall and ROC-AUC are used later.
- **Churn by contract:** month-to-month customers churn at roughly 43%, compared with about 11% for one-year and about 3% for two-year contracts.
- **Tenure:** churners are concentrated in the first few months, while customers with a long tenure rarely leave.
- **Monthly charges:** customers who churned pay more per month, with a median of roughly 80 compared with roughly 65 for those who stayed.

6.6 Feature Engineering and Train/Test Split

1.5 Feature engineering

We create new columns that capture customer behaviour better than the raw columns:

New feature	Meaning
tenure_group	Customer age bucket: 0-12, 13-24, 25-48, 48+ months
avg_monthly_spend	TotalCharges / tenure - what the customer has paid per month on average
charge_diff	Current MonthlyCharges minus average spend (positive = bill has risen)
num_services	How many services the customer subscribes to
is_autopay	1 if paying by automatic bank transfer or credit card
has_family	1 if the customer has a partner or dependents

Yes/No columns are converted to 1/0 and multi-category columns are one-hot encoded. All of this happens

```
[ ]
y = (df["Churn"] == "Yes").astype(int)
X_raw = df.drop(columns="Churn")

X = prepare(X_raw)
print("Shape after cleaning, feature engineering and encoding:", X.shape)

new_feats = ["avg_monthly_spend", "charge_diff", "num_services", "is_autopay", "has_family"]
print("\nAverage of engineered features, split by churn (0 = stayed, 1 = churned):")
display(X[new_feats].groupby(y).mean().round(2))
X.head()
```

Figure 15: Target encoding, applying prepare() and comparing engineered features by churn

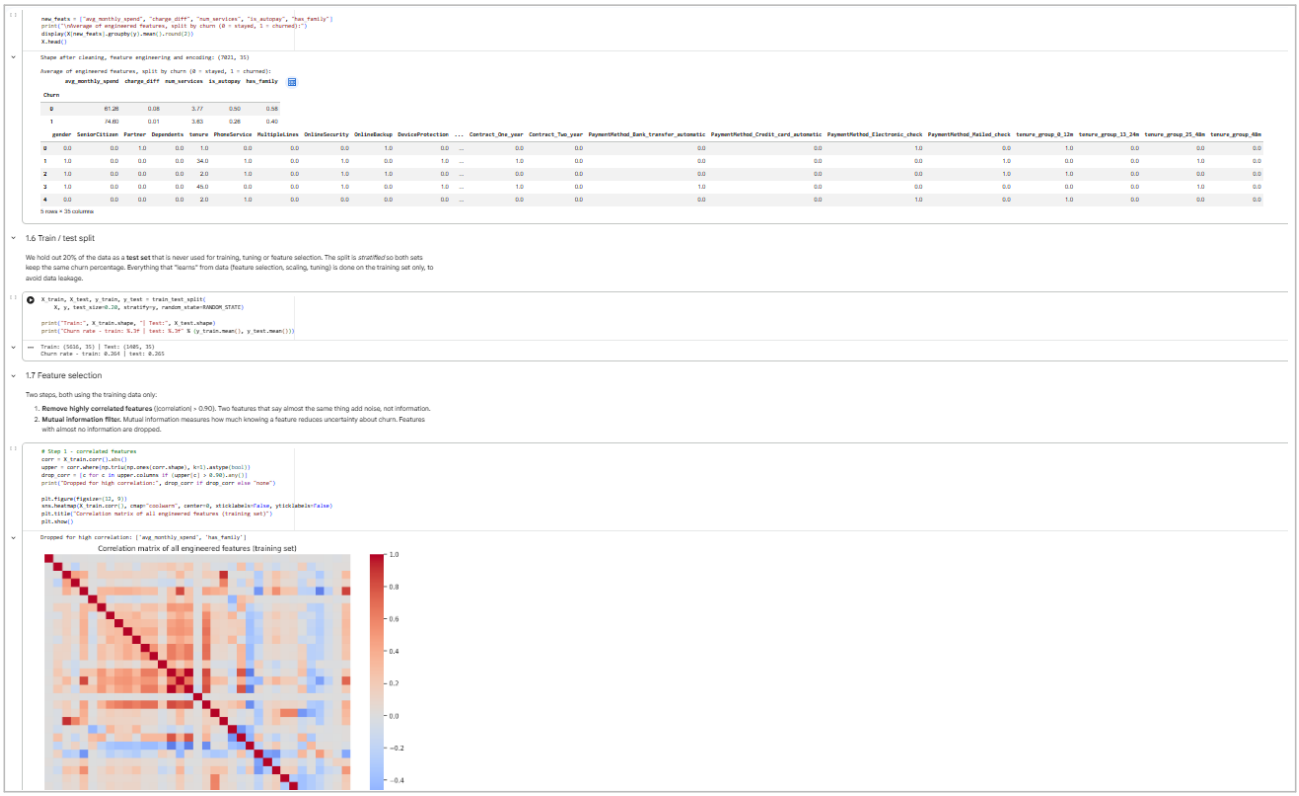


Figure 16: Engineered feature averages, the train/test split and the correlation heat map

The target was encoded as 1 for churn and 0 for stay, and prepare() was applied to the cleaned data. After cleaning, encoding and adding six engineered features (tenure_group, avg_monthly_spend, charge_diff, num_services, is_autopay and has_family), the feature matrix has **7,021 rows and 35**

columns.

Comparing the engineered features by outcome shows useful signals. Customers who churned had a higher average monthly spend (74.80 against 61.28), were much less likely to pay by automatic transfer or credit card (26% against 50%), and were less likely to have a partner or dependents (40% against 58%).

The data was then split 80/20 using stratification, giving **5,616 training rows and 1,405 test rows**. The churn rate is 26.4% in the training set and 26.5% in the test set, confirming that the class proportions were preserved. The test set is not used for feature selection, scaling or tuning, so it gives an honest final evaluation.

6.7 Feature Selection and Data Scaling

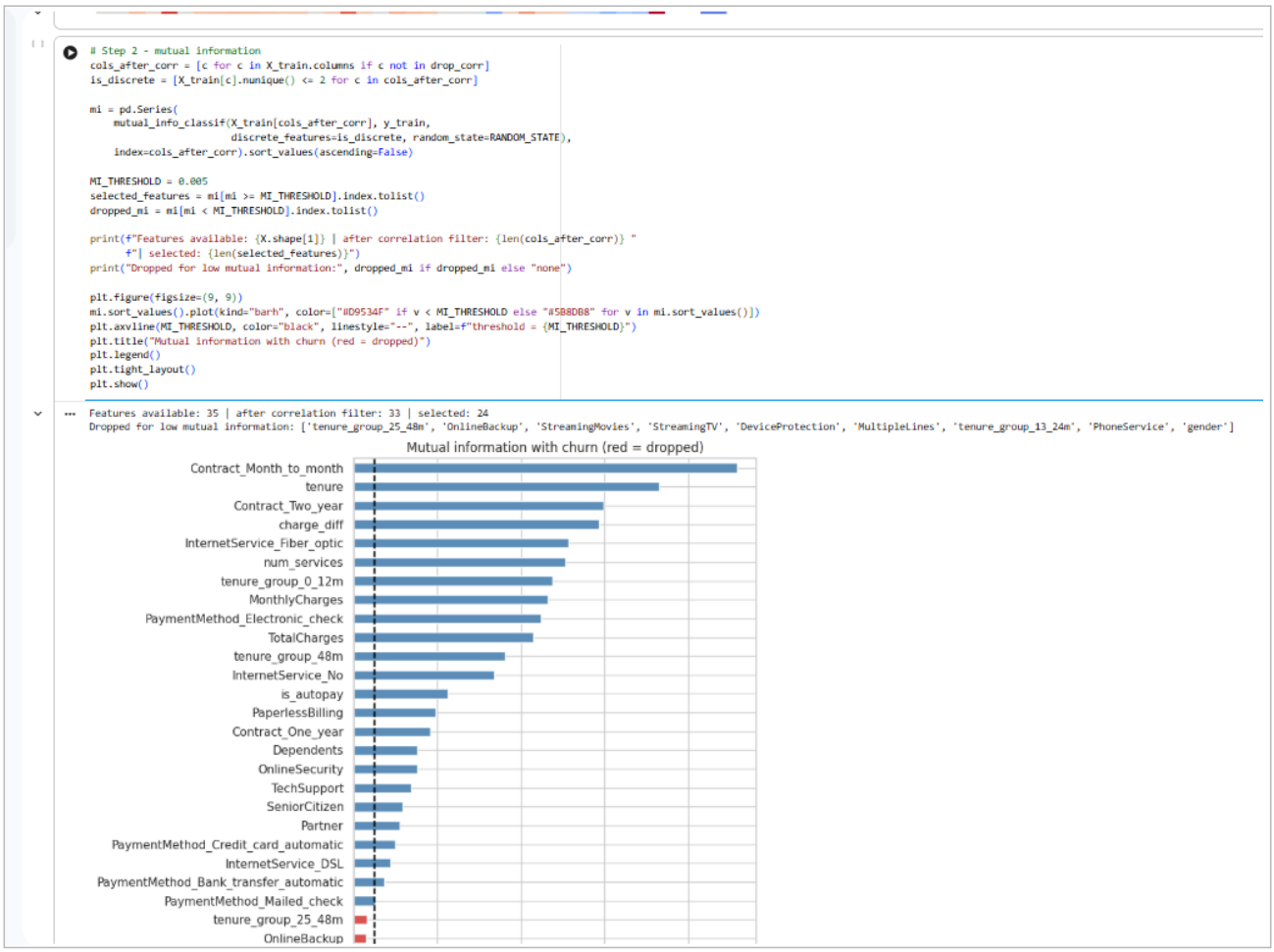


Figure 17: Mutual information feature selection: code, selected features and bar chart

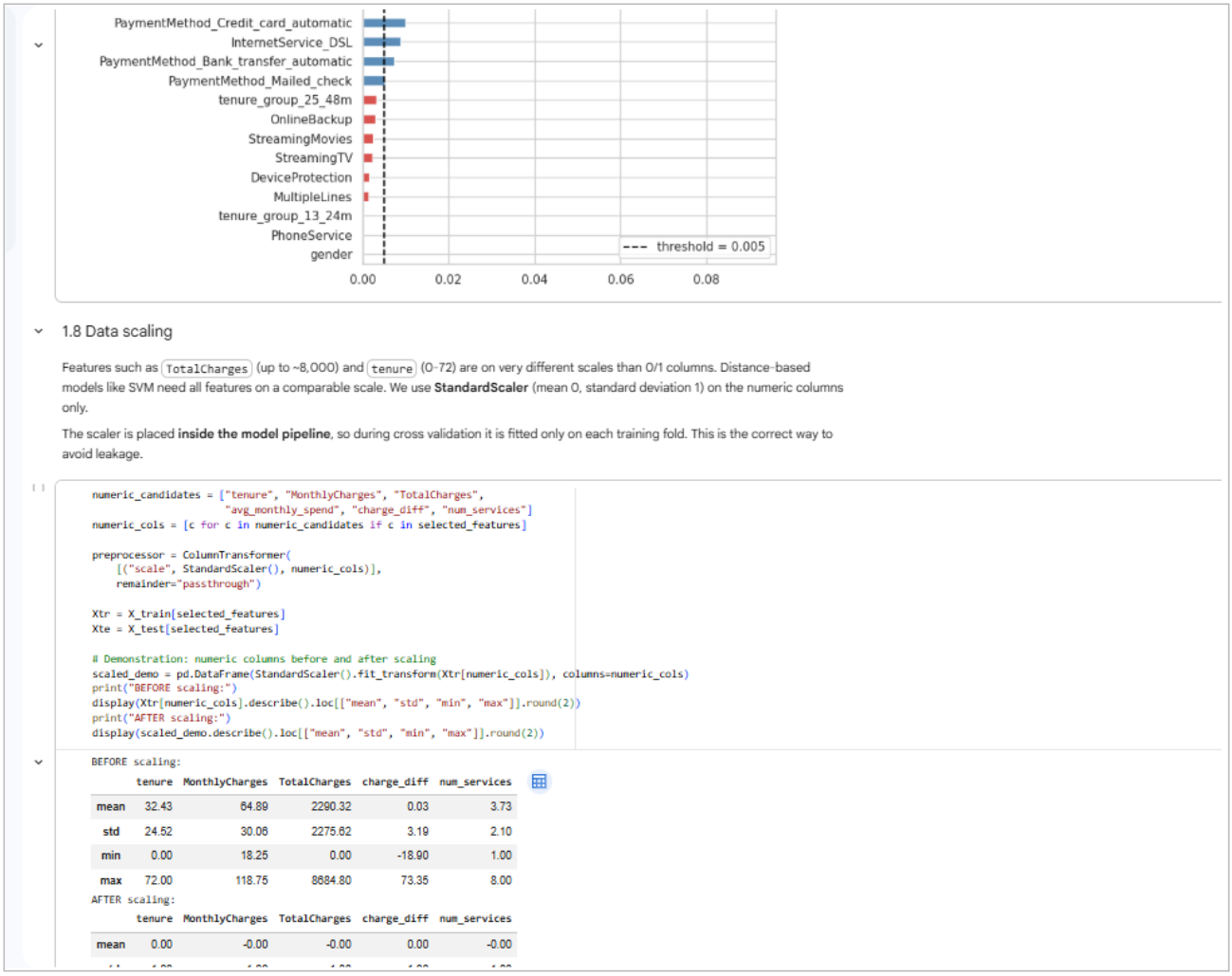


Figure 18: Remaining mutual information bars and the StandardScaler before/after statistics

Feature selection used the training data only and had two steps. First, a correlation filter with a threshold of 0.90 (heat map in Figure 16) removed **avg_monthly_spend** and **has_family**, because each is more than 90% correlated with another feature and adds almost no new information.

Second, the mutual information between each of the remaining 33 features and the churn target was calculated, and features scoring below 0.005 were removed. This dropped nine low-information features: gender, PhoneService, MultipleLines, StreamingTV, StreamingMovies, DeviceProtection, OnlineBackup, tenure_group_13_24m and tenure_group_25_48m. The **24 selected features** are led by Contract_Month_to_month, tenure, Contract_Two_year, charge_diff and InternetService_Fiber_optic, which agrees with the patterns found during the exploratory analysis. The engineered feature charge_diff ranks fourth, which suggests that it adds information beyond the original columns.

The numeric features are on very different scales (for example tenure from 0 to 72, but TotalCharges up to 8,684.80). **StandardScaler** is therefore applied to the five numeric columns that survived selection (tenure, MonthlyCharges, TotalCharges, charge_diff and num_services), while the 0/1 columns pass through unchanged. The before/after statistics show the effect: after scaling, each column has a mean of 0 and a standard deviation of 1. In the modelling pipeline the scaler is fitted on training data only (inside each cross-validation fold), which prevents data leakage.

7. Part 2 - Model Development

Part 2 - Model Development

We train five different models, each inside the same pipeline (scaling then model):

Model	Idea in one line
Decision Tree	A flowchart of yes/no questions about the customer
Random Forest	Many decision trees voting together
Support Vector Machine	Finds the best boundary separating churners from stayers
Gradient Boosting	Trees built one after another, each fixing the previous one's mistakes
XGBoost (optional)	A faster, regularised version of gradient boosting

Because only ~27% of customers churn, we use `class_weight="balanced"` where available so the models do not simply predict "stays" for everyone.

Metrics: Accuracy alone is misleading on imbalanced data, so we also report Precision, Recall, F1 and ROC-AUC. For churn, **Recall** matters (how many real churners we catch) and **ROC-AUC** measures how well the model ranks risky customers above safe ones.

```
neg, pos = np.bincount(y_train)
scale_pos_weight = neg / pos

models = {
    "Decision Tree": DecisionTreeClassifier(class_weight="balanced", random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=200, class_weight="balanced",
                                            random_state=RANDOM_STATE, n_jobs=-1),
    "SVM": SVC(kernel="rbf", class_weight="balanced", random_state=RANDOM_STATE),
    "Gradient Boosting": GradientBoostingClassifier(random_state=RANDOM_STATE),
}
if HAS_XGB:
    models["XGBoost"] = XGBClassifier(eval_metric="logloss", scale_pos_weight=scale_pos_weight,
                                      random_state=RANDOM_STATE, n_jobs=-1)

def make_pipe(model):
    return Pipeline([("prep", clone(preprocessor)), ("model", clone(model))])

def get_scores(pipe, X_):
    """Probability of churn if available, otherwise the SVM decision score."""
    if hasattr(pipe, "predict_proba"):
        return pipe.predict_proba(X_)[:, 1]
    return pipe.decision_function(X_)

def evaluate_on(pipe, X_, y_):
    pred = pipe.predict(X_)
    return {"Accuracy": accuracy_score(y_, pred),
            "Precision": precision_score(y_, pred),
            "Recall": recall_score(y_, pred),
            "F1": f1_score(y_, pred),
            "ROC-AUC": roc_auc_score(y_, get_scores(pipe, X_))}

cv5 = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

baseline_rows, baseline_pipes = [], {}
for name, model in models.items():
    t0 = time.time()
    pipe = make_pipe(model).fit(Xtr, y_train)
    row = evaluate_on(pipe, Xte, y_test)
    row["CV ROC-AUC (train)"] = cross_val_score(pipe, Xtr, y_train, cv=cv5,
                                                scoring="roc_auc", n_jobs=-1).mean()
    row["Model"] = name
    baseline_rows.append(row)
```

Figure 19: Model definitions, pipeline helper, evaluation functions and the training loop

Five models were trained, each inside an identical pipeline of scaling followed by the classifier: **Decision Tree, Random Forest, Support Vector Machine (RBF kernel), Gradient Boosting** and **XGBoost** (the optional model). Because only about a quarter of customers churn, class_weight="balanced" is used for the Decision Tree, Random Forest and SVM, and scale_pos_weight is used for XGBoost, so that the models do not simply predict "stay" for everyone. Gradient Boosting has no class-weight option and uses its default settings.

Helper functions evaluate each model on the held-out test set with Accuracy, Precision, Recall, F1 and ROC-AUC, and also report a 5-fold cross-validated ROC-AUC on the training data.

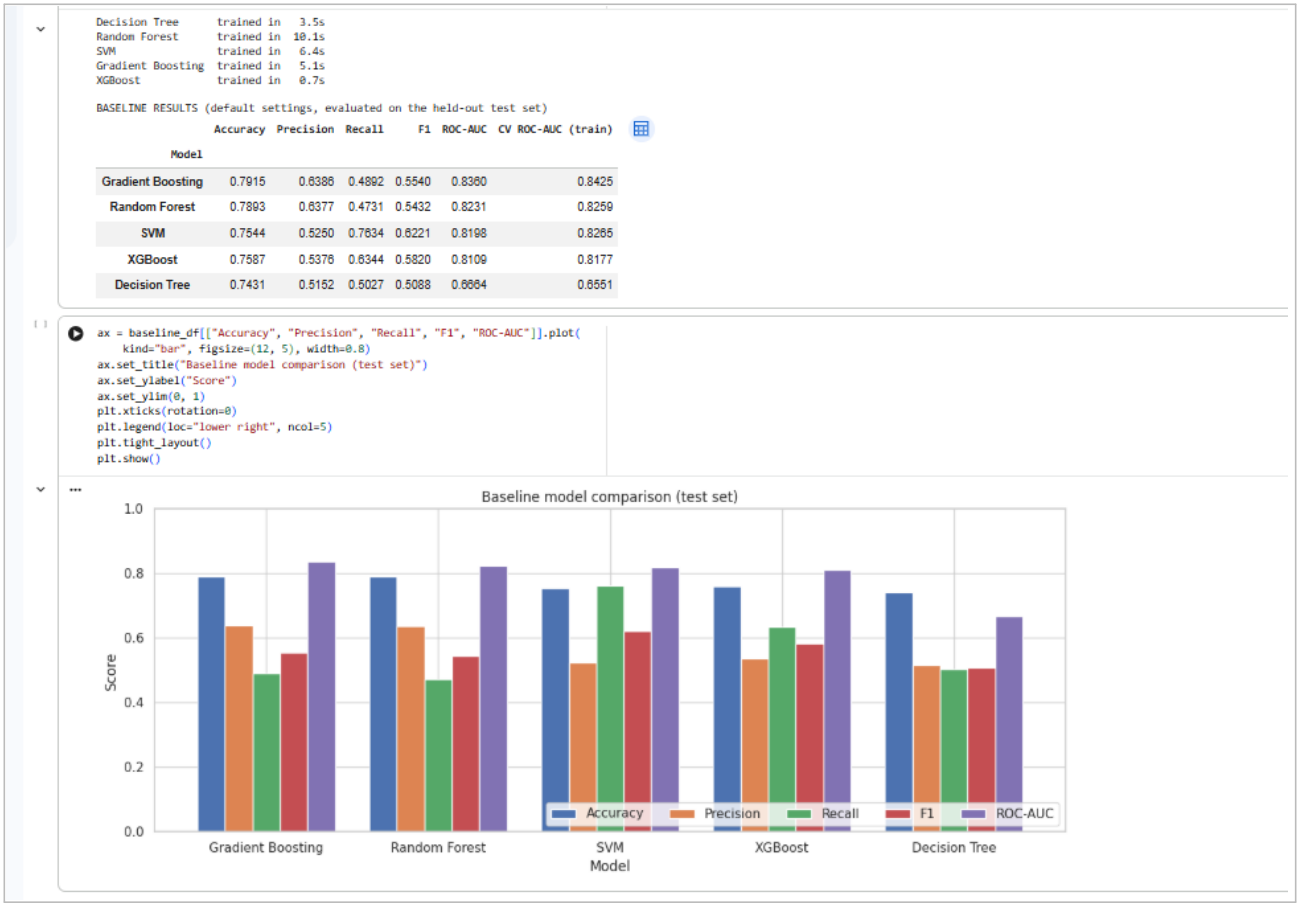


Figure 20: Baseline results on the test set and the baseline comparison chart

Table 1 repeats the baseline results shown in Figure 20. Training times were 0.7 s for XGBoost, 3.5 s for the Decision Tree, 5.1 s for Gradient Boosting, 6.4 s for the SVM and 10.1 s for the Random Forest.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC (train)
Gradient Boosting	0.7915	0.6386	0.4892	0.5540	0.8360	0.8425
Random Forest	0.7893	0.6377	0.4731	0.5432	0.8231	0.8259
SVM	0.7544	0.5250	0.7634	0.6221	0.8198	0.8265
XGBoost	0.7587	0.5376	0.6344	0.5820	0.8109	0.8177
Decision Tree	0.7431	0.5152	0.5027	0.5088	0.6664	0.6551

Table 1: Baseline model results (default settings, test set)

Observations from the baseline comparison:

- **Gradient Boosting** has the best baseline ROC-AUC (0.8360 on the test set and 0.8425 in cross validation), followed by Random Forest (0.8231) and SVM (0.8198).
- The **SVM** has by far the highest recall (0.7634): it identifies about 76% of the customers who actually churn, at the cost of lower precision (0.5250). XGBoost sits in between, with recall 0.6344 and F1 0.5820.

- Gradient Boosting and Random Forest have the highest accuracy (about 79%) and precision (about 64%) but miss more churners, with recall of only 0.49 and 0.47.
- The **Decision Tree** is clearly the weakest (ROC-AUC 0.6664). This is expected for an untuned single tree, which tends to overfit, and tuning in Part 3 should help.
- Accuracy lies between 74% and 79% for every model, yet a model that predicted "stay" for every customer would already score about 73.5%. This confirms that recall, F1 and ROC-AUC are more informative than accuracy for this problem.

8. Part 3 - Model Optimization

8.1 Hyperparameter Tuning

Part 3 - Model Optimization

3.1 Hyperparameter tuning

Every model has settings (hyperparameters) that are not learned from data, such as tree depth. We search for the best ones using **cross-validated search on the training set**:

- **GridSearchCV** for the Decision Tree - its search space is small, so we try every combination.
- **RandomizedSearchCV** for the other models - their search spaces are large, so we sample `N_ITER` random combinations.

Each candidate is scored with 5-fold cross validation using ROC-AUC. The test set is not touched.

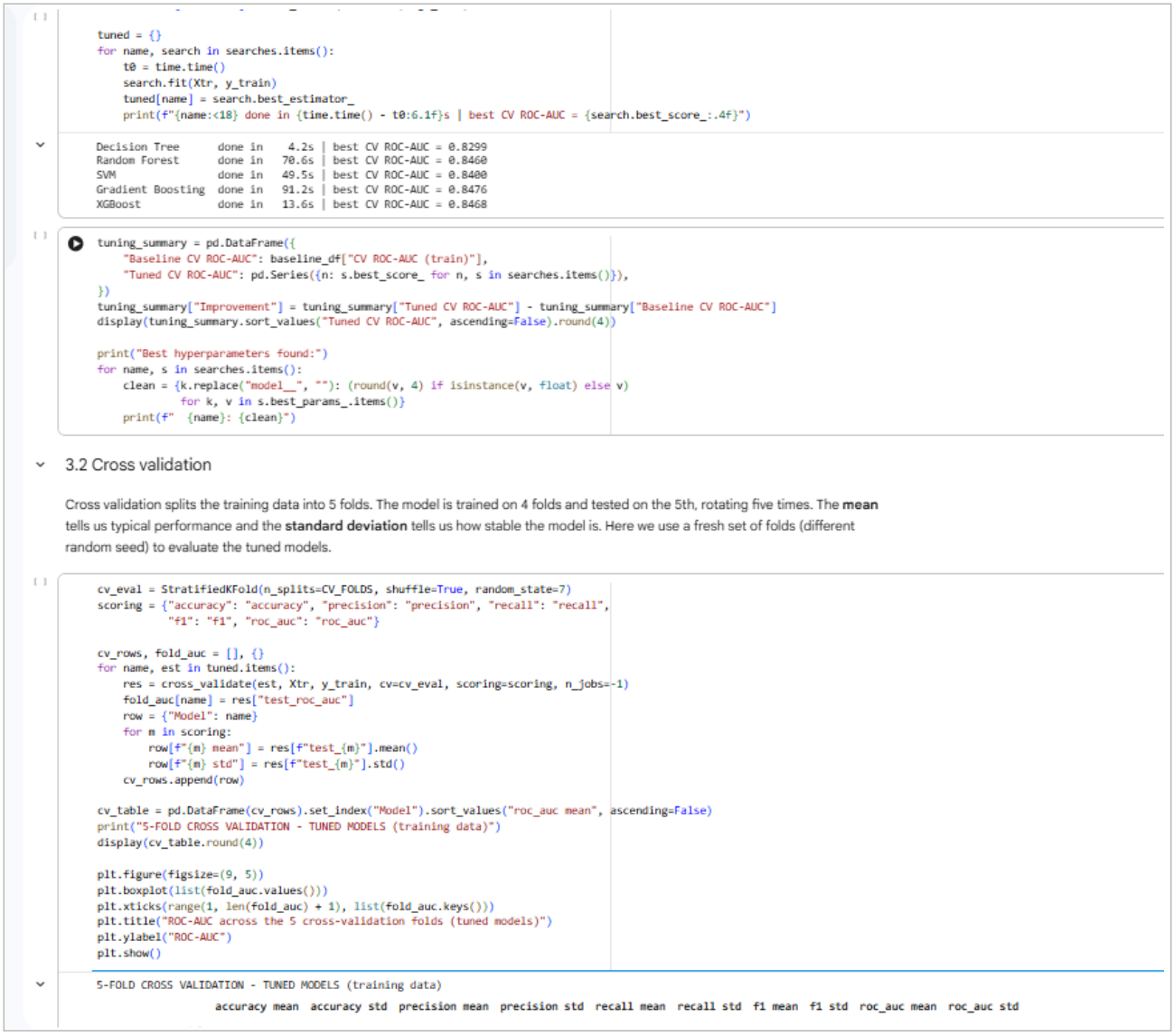
```
dt_grid = {
    "model__max_depth": [3, 5, 7, 10, None],
    "model__min_samples_leaf": [1, 5, 10, 20],
    "model__criterion": ["gini", "entropy"],
}
rf_dist = {
    "model__n_estimators": randint(100, 400),
    "model__max_depth": [None, 5, 8, 12, 16],
    "model__min_samples_leaf": randint(1, 15),
    "model__min_samples_split": randint(2, 20),
    "model__max_features": ["sqrt", "log2", 0.5],
}
svm_dist = {
    "model__C": loguniform(0.1, 50),
    "model__gamma": loguniform(1e-4, 1e-1),
}
gb_dist = {
    "model__n_estimators": randint(100, 400),
    "model__learning_rate": loguniform(0.01, 0.3),
    "model__max_depth": randint(2, 5),
    "model__subsample": uniform(0.6, 0.4),
    "model__min_samples_leaf": randint(1, 20),
}
xgb_dist = {
    "model__n_estimators": randint(100, 500),
    "model__learning_rate": loguniform(0.01, 0.3),
    "model__max_depth": randint(2, 7),
    "model__subsample": uniform(0.6, 0.4),
    "model__colsample_bytree": uniform(0.6, 0.4),
}

def rand_search(name, dist):
    return RandomizedSearchCV(make_pipe(models[name]), dist, n_iter=N_ITER, scoring="roc_auc",
                              cv=cv5, n_jobs=-1, random_state=RANDOM_STATE, refit=True)

searches = {
    "Decision Tree": GridSearchCV(make_pipe(models["Decision Tree"]), dt_grid,
                                  scoring="roc_auc", cv=cv5, n_jobs=-1, refit=True),
    "Random Forest": rand_search("Random Forest", rf_dist),
    "SVM": rand_search("SVM", svm_dist),
    "Gradient Boosting": rand_search("Gradient Boosting", gb_dist),
}
if HAS_XGB:
    searches["XGBoost"] = rand_search("XGBoost", xgb_dist)

tuned = {}
for name, search in searches.items():
    t0 = time.time()
    search.fit(X_train, y_train)
```

Figure 21: Search spaces and the tuning set-up (GridSearchCV and RandomizedSearchCV)



```
tuned = {}
for name, search in searches.items():
    t0 = time.time()
    search.fit(Xtr, y_train)
    tuned[name] = search.best_estimator_
    print(f"{name:<18} done in {time.time() - t0:6.1f}s | best CV ROC-AUC = {search.best_score_:.4f}")
```

```
Decision Tree      done in    4.2s | best CV ROC-AUC = 0.8299
Random Forest      done in   70.6s | best CV ROC-AUC = 0.8460
SVM                done in   49.5s | best CV ROC-AUC = 0.8400
Gradient Boosting  done in   91.2s | best CV ROC-AUC = 0.8476
XGBoost            done in   13.6s | best CV ROC-AUC = 0.8468
```

```
tuning_summary = pd.DataFrame({
    "Baseline CV ROC-AUC": baseline_df["CV ROC-AUC (train)"],
    "Tuned CV ROC-AUC": pd.Series({n: s.best_score_ for n, s in searches.items()}),
})
tuning_summary["Improvement"] = tuning_summary["Tuned CV ROC-AUC"] - tuning_summary["Baseline CV ROC-AUC"]
display(tuning_summary.sort_values("Tuned CV ROC-AUC", ascending=False).round(4))

print("Best hyperparameters found:")
for name, s in searches.items():
    clean = {k.replace("model__", ""): (round(v, 4) if isinstance(v, float) else v)
             for k, v in s.best_params_.items()}
    print(f"  {name}: {clean}")
```

3.2 Cross validation

Cross validation splits the training data into 5 folds. The model is trained on 4 folds and tested on the 5th, rotating five times. The **mean** tells us typical performance and the **standard deviation** tells us how stable the model is. Here we use a fresh set of folds (different random seed) to evaluate the tuned models.

```
cv_eval = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=7)
scoring = {"accuracy": "accuracy", "precision": "precision", "recall": "recall",
           "f1": "f1", "roc_auc": "roc_auc"}

cv_rows, fold_auc = [], {}
for name, est in tuned.items():
    res = cross_validate(est, Xtr, y_train, cv=cv_eval, scoring=scoring, n_jobs=-1)
    fold_auc[name] = res["test_roc_auc"]
    row = {"Model": name}
    for m in scoring:
        row[f"{m} mean"] = res[f"test_{m}"].mean()
        row[f"{m} std"] = res[f"test_{m}"].std()
    cv_rows.append(row)

cv_table = pd.DataFrame(cv_rows).set_index("Model").sort_values("roc_auc mean", ascending=False)
print("5-FOLD CROSS VALIDATION - TUNED MODELS (training data)")
display(cv_table.round(4))

plt.figure(figsize=(9, 5))
plt.boxplot(list(fold_auc.values()))
plt.xticks(range(1, len(fold_auc) + 1), list(fold_auc.keys()))
plt.title("ROC-AUC across the 5 cross-validation folds (tuned models)")
plt.ylabel("ROC-AUC")
plt.show()
```

```
5-FOLD CROSS VALIDATION - TUNED MODELS (training data)
```

	accuracy mean	accuracy std	precision mean	precision std	recall mean	recall std	f1 mean	f1 std	roc_auc mean	roc_auc std

Figure 22: Tuning results for each model and the start of the cross-validation step

Every model has settings, called hyperparameters, that are not learned from the data (for example the maximum depth of a tree). These were tuned with a cross-validated search on the **training set only**, so the test set stayed untouched. Each candidate was scored with 5-fold stratified cross validation using ROC-AUC.

- **Decision Tree:** GridSearchCV tried all 40 combinations of max_depth, min_samples_leaf and criterion, because the search space is small.
- **Random Forest:** RandomizedSearchCV (15 random combinations) over n_estimators, max_depth, min_samples_leaf, min_samples_split and max_features.
- **SVM:** RandomizedSearchCV over the regularization strength C and the kernel width gamma.
- **Gradient Boosting and XGBoost:** RandomizedSearchCV over n_estimators, learning_rate, max_depth, subsample and either min_samples_leaf (Gradient Boosting) or colsample_bytree (XGBoost).

The best cross-validated ROC-AUC scores found were 0.8299 for the Decision Tree, 0.8460 for Random Forest, 0.8400 for the SVM, 0.8476 for Gradient Boosting and 0.8468 for XGBoost. The searches took 4.2 s, 70.6 s, 49.5 s, 91.2 s and 13.6 s respectively. Table 2 compares these scores

with the baseline cross-validated scores from Part 2.

Model	Baseline CV ROC-AUC	Tuned CV ROC-AUC	Improvement
Gradient Boosting	0.8425	0.8476	+0.0051
XGBoost	0.8177	0.8468	+0.0291
Random Forest	0.8259	0.8460	+0.0201
SVM	0.8265	0.8400	+0.0135
Decision Tree	0.6551	0.8299	+0.1748

Table 2: Effect of hyperparameter tuning on cross-validated ROC-AUC (training data)

Tuning helped every model. The Decision Tree improved the most (from 0.6551 to 0.8299), because limiting its depth and leaf size stops it from memorising the training data. The ensemble models were already strong, so their gains were smaller (between +0.005 and +0.029). After tuning, all five models reach a cross-validated ROC-AUC between about 0.83 and 0.85.

8.2 Cross Validation

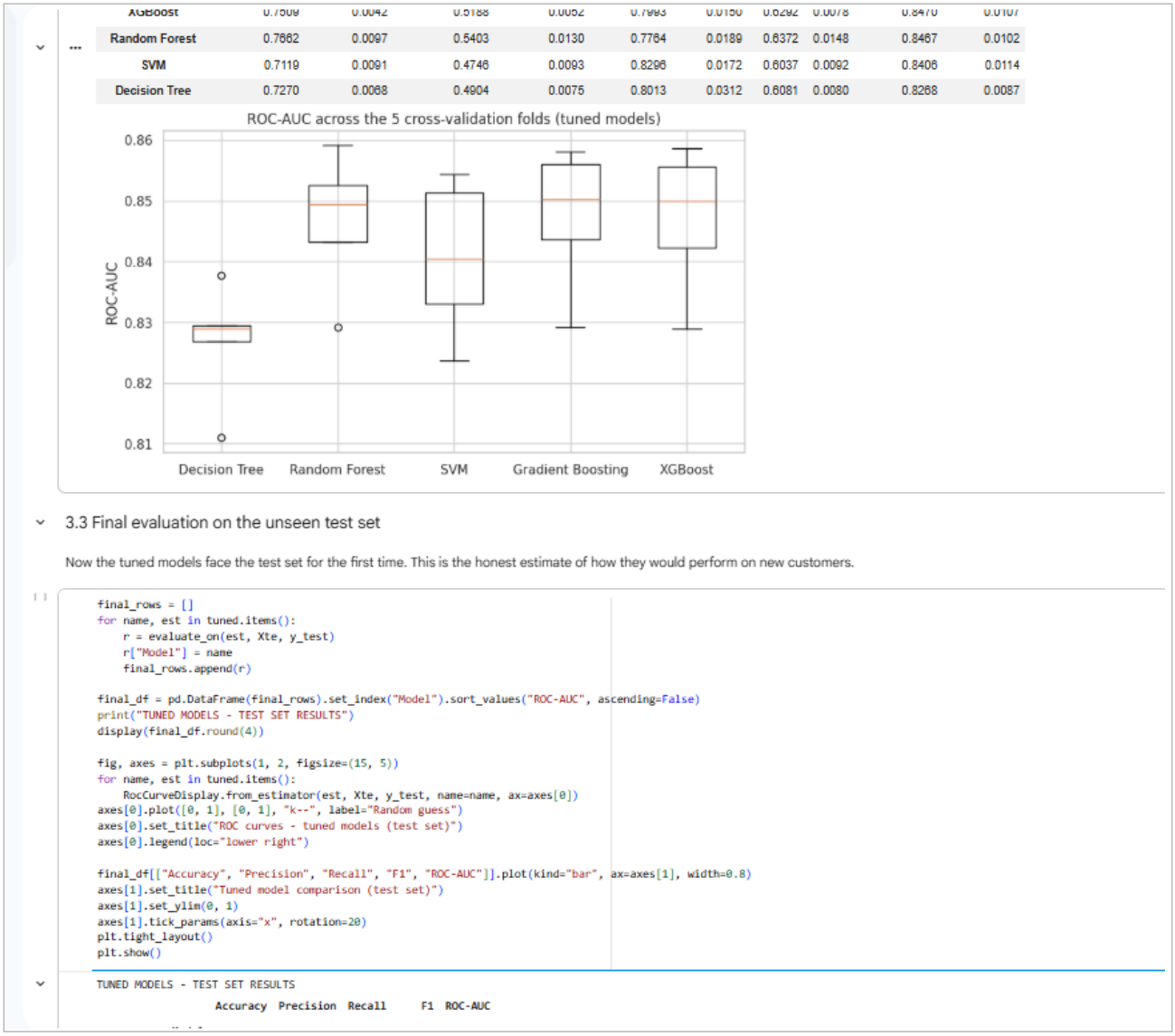


Figure 23: Cross-validation results table and ROC-AUC across the five folds (tuned models)

To judge the tuned models fairly, they were evaluated again with a fresh set of 5 folds (a different random seed). For each model the notebook reports the mean and the standard deviation of accuracy, precision, recall, F1 and ROC-AUC. The mean tells us the typical performance and the standard deviation tells us how stable the model is.

The box plot shows the ROC-AUC of every fold. The Random Forest, Gradient Boosting and XGBoost boxes sit highest, at roughly 0.85, the SVM is slightly lower, and the Decision Tree is lowest, at roughly 0.83. The standard deviation is about 0.01 for every model (0.0104 for Gradient Boosting), so the results are stable from fold to fold and the differences between the top three models are small compared with this natural variation.

8.3 Final Evaluation on the Unseen Test Set

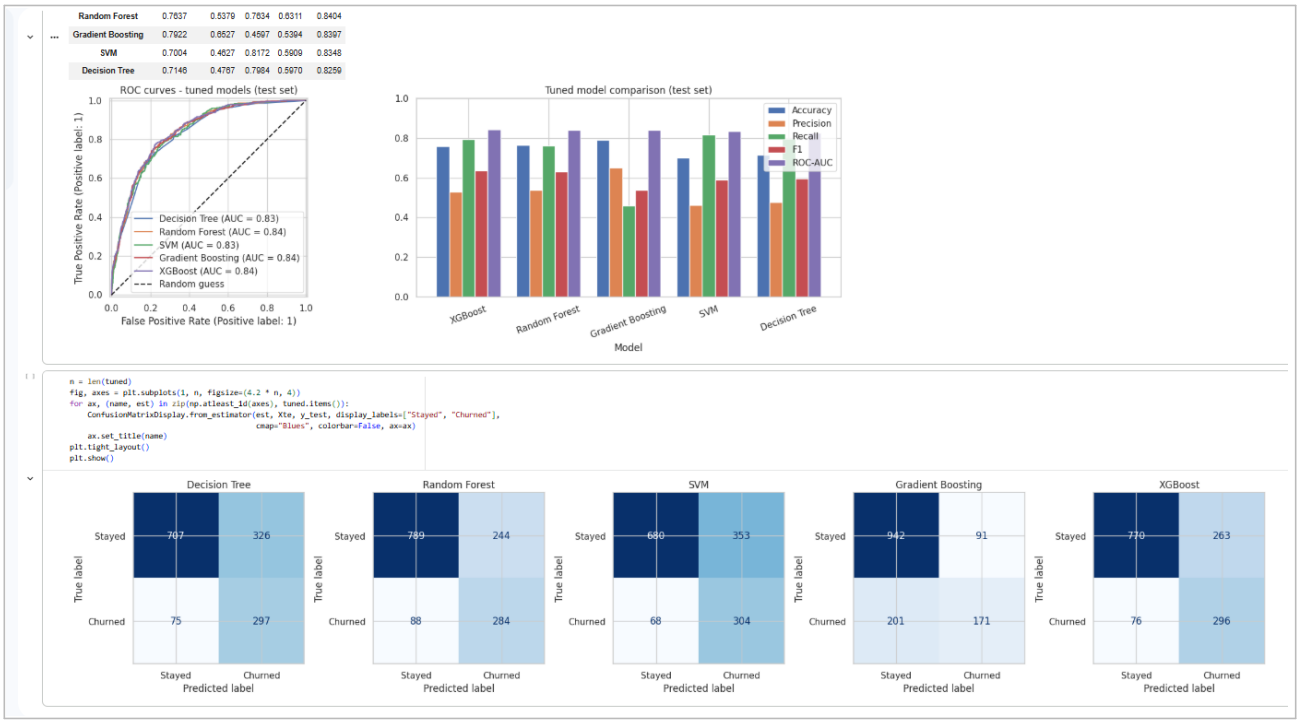


Figure 24: Tuned models on the test set: results table, ROC curves, metric comparison and confusion matrices

After tuning, the models were evaluated on the test set for the first time. Table 3 gives their accuracy, precision, recall and F1, calculated from the confusion matrices in Figure 24; the ROC-AUC is read from the ROC-curve legend.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	0.7587	0.5295	0.7957	0.6359	0.84
Random Forest	0.7637	0.5379	0.7634	0.6311	0.84
Gradient Boosting	0.7922	0.6527	0.4597	0.5394	0.84
SVM	0.7004	0.4627	0.8172	0.5909	0.83
Decision Tree	0.7146	0.4767	0.7984	0.5970	0.83

Table 3: Tuned model results on the test set (1,405 customers, of whom 372 churned)

- **ROC-AUC improved for every model** compared with the baseline: Decision Tree 0.67 to 0.83, Random Forest 0.82 to 0.84, SVM 0.82 to 0.83, XGBoost 0.81 to 0.84, and Gradient Boosting 0.836 to 0.840.
- The five ROC curves almost overlap (AUC 0.83 to 0.84). After tuning, the models rank customers by churn risk in a very similar way, and the main differences between them are in where each model places its decision threshold.
- The **SVM** catches the most churners (304 of 372, recall 0.817) but also raises the most false alarms (353). The Decision Tree (297 caught), XGBoost (296) and Random Forest (284) behave similarly, with recall between 0.76 and 0.80.
- **Gradient Boosting** is the most cautious model. It catches only 171 of the 372 churners (recall 0.460), but it has the fewest false alarms (91), the highest precision (0.653) and the highest accuracy (0.792). It is the only model without class balancing, which explains this behaviour.

8.4 Choosing the Best Model

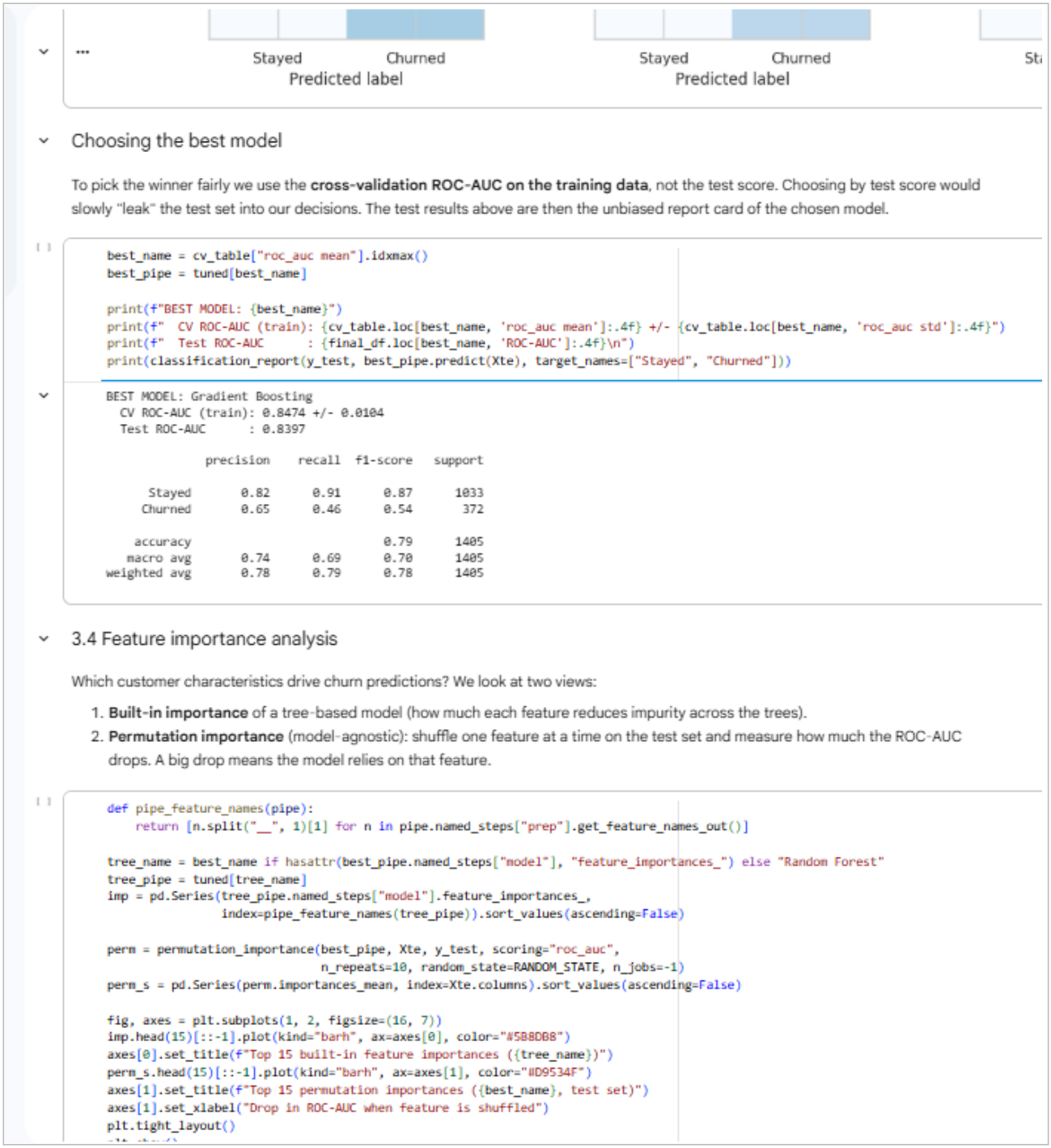


Figure 25: Selecting the best model and its classification report (top); start of the feature importance code

The best model was chosen by its **cross-validated ROC-AUC on the training data** rather than by its test score, because choosing by test score would slowly leak the test set into the decision. On this basis the winner is **Gradient Boosting**, with a cross-validated ROC-AUC of 0.8474 (+/- 0.0104) and a test ROC-AUC of 0.8397.

The classification report shows that the model recognises loyal customers very well (precision 0.82, recall 0.91) but identifies fewer than half of the churners (recall 0.46, precision 0.65) at the default threshold of 0.5. Overall accuracy is 0.79 on 1,405 test customers.

It is worth being honest about this choice: XGBoost and Random Forest are within about 0.001 of Gradient Boosting in cross-validated ROC-AUC, far less than the fold-to-fold standard deviation of

about 0.01, so the top three models are effectively tied. If the business cares most about catching as many churners as possible, a high-recall model (for example the SVM or Random Forest) or a lower decision threshold for Gradient Boosting would be a better practical choice.

8.5 Feature Importance Analysis

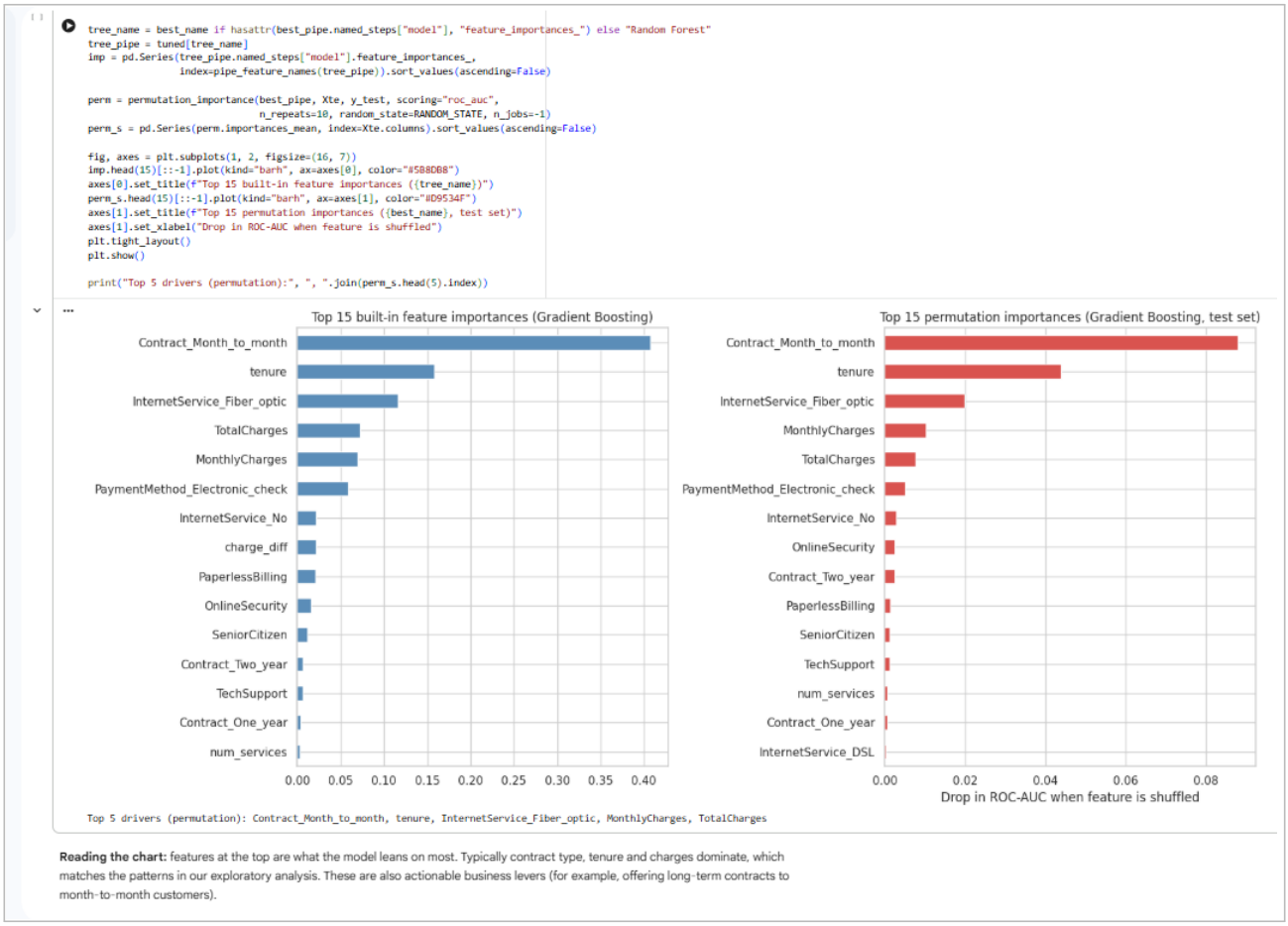


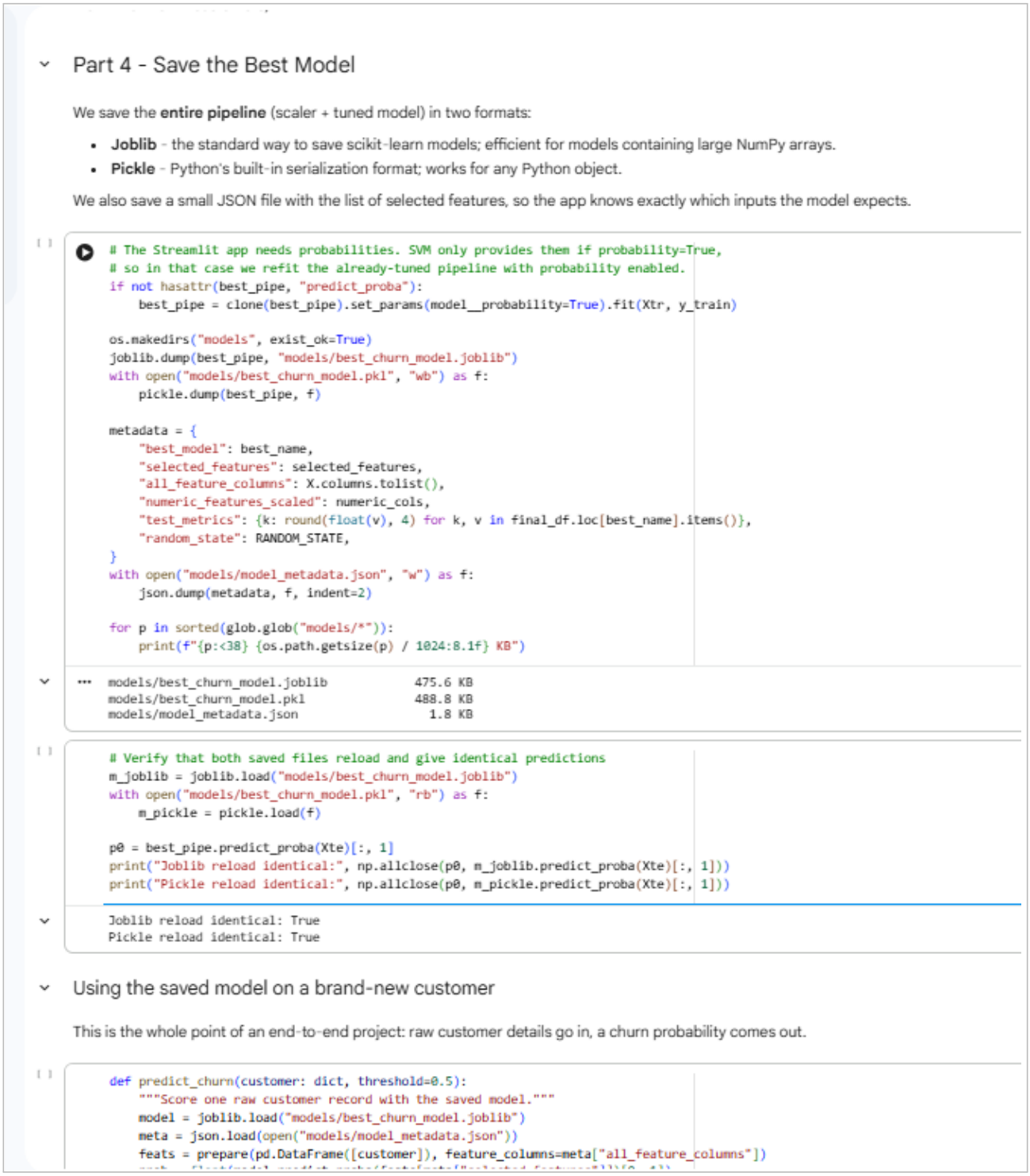
Figure 26: Top 15 built-in importances and top 15 permutation importances for Gradient Boosting

Two views of feature importance were produced for the best model. The **built-in importance** measures how much each feature reduces impurity across the trees. The **permutation importance** shuffles one feature at a time on the test set and measures how much the ROC-AUC drops, so a large drop means the model relies heavily on that feature.

Both views agree on the top three drivers of churn: **Contract_Month_to_month** (built-in importance about 0.41, ROC-AUC drop of about 0.09 when shuffled), **tenure** (about 0.16 and 0.04) and **InternetService_Fiber_optic** (about 0.12 and 0.02). The top five by permutation importance are Contract_Month_to_month, tenure, InternetService_Fiber_optic, MonthlyCharges and TotalCharges. PaymentMethod_Electronic_check also appears in the top six of both charts.

This matches the patterns found in the exploratory analysis. For the business, the clearest levers are encouraging month-to-month customers to move to longer contracts and paying attention to new customers during their first months. These features show what predicts churn, not necessarily what causes it. The engineered feature charge_diff is in the built-in top ten but is not among the top 15 permutation importances, which suggests that its information overlaps with MonthlyCharges and TotalCharges.

9. Part 4 - Save the Best Model



Part 4 - Save the Best Model

We save the **entire pipeline** (scaler + tuned model) in two formats:

- **Joblib** - the standard way to save scikit-learn models; efficient for models containing large NumPy arrays.
- **Pickle** - Python's built-in serialization format; works for any Python object.

We also save a small JSON file with the list of selected features, so the app knows exactly which inputs the model expects.

```
# The Streamlit app needs probabilities. SVM only provides them if probability=True,
# so in that case we refit the already-tuned pipeline with probability enabled.
if not hasattr(best_pipe, "predict_proba"):
    best_pipe = clone(best_pipe).set_params(model__probability=True).fit(Xtr, y_train)

os.makedirs("models", exist_ok=True)
joblib.dump(best_pipe, "models/best_churn_model.joblib")
with open("models/best_churn_model.pkl", "wb") as f:
    pickle.dump(best_pipe, f)

metadata = {
    "best_model": best_name,
    "selected_features": selected_features,
    "all_feature_columns": X.columns.tolist(),
    "numeric_features_scaled": numeric_cols,
    "test_metrics": {k: round(float(v), 4) for k, v in final_df.loc[best_name].items()},
    "random_state": RANDOM_STATE,
}
with open("models/model_metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)

for p in sorted(glob.glob("models/*")):
    print(f"{p:<38} {os.path.getsize(p) / 1024:8.1f} KB")
```

```
models/best_churn_model.joblib          475.6 KB
models/best_churn_model.pkl             488.8 KB
models/model_metadata.json                1.8 KB
```

```
# Verify that both saved files reload and give identical predictions
m_joblib = joblib.load("models/best_churn_model.joblib")
with open("models/best_churn_model.pkl", "rb") as f:
    m_pickle = pickle.load(f)

p0 = best_pipe.predict_proba(Xte)[:, 1]
print("Joblib reload identical:", np.allclose(p0, m_joblib.predict_proba(Xte)[:, 1]))
print("Pickle reload identical:", np.allclose(p0, m_pickle.predict_proba(Xte)[:, 1]))
```

```
Joblib reload identical: True
Pickle reload identical: True
```

Using the saved model on a brand-new customer

This is the whole point of an end-to-end project: raw customer details go in, a churn probability comes out.

```
def predict_churn(customer: dict, threshold=0.5):
    """Score one raw customer record with the saved model."""
    model = joblib.load("models/best_churn_model.joblib")
    meta = json.load(open("models/model_metadata.json"))
    feats = prepare(pd.DataFrame([customer]), feature_columns=meta["all_feature_columns"])
```

Figure 27: Saving the model with Joblib and Pickle, and verifying that both files reload correctly

The **entire pipeline** (the scaler together with the tuned Gradient Boosting model) was saved in two formats. **Joblib** is the standard way to save scikit-learn models and handles large NumPy arrays efficiently, while **Pickle** is Python's built-in serialization format. Saving the whole pipeline, not just the classifier, means that new data is scaled in exactly the same way as the training data. A small JSON file was also saved with the model name, the list of selected features, the list of all feature columns, the scaled numeric columns, the test metrics and the random seed, so that the prediction app knows

exactly which inputs the model expects.

The three files are best_churn_model.joblib (about 476 KB), best_churn_model.pkl (about 489 KB) and model_metadata.json (about 2 KB). As a check, both model files were loaded back from disk and used to predict the test set. Both reloaded models returned exactly the same churn probabilities as the original model ("Joblib reload identical: True" and "Pickle reload identical: True").

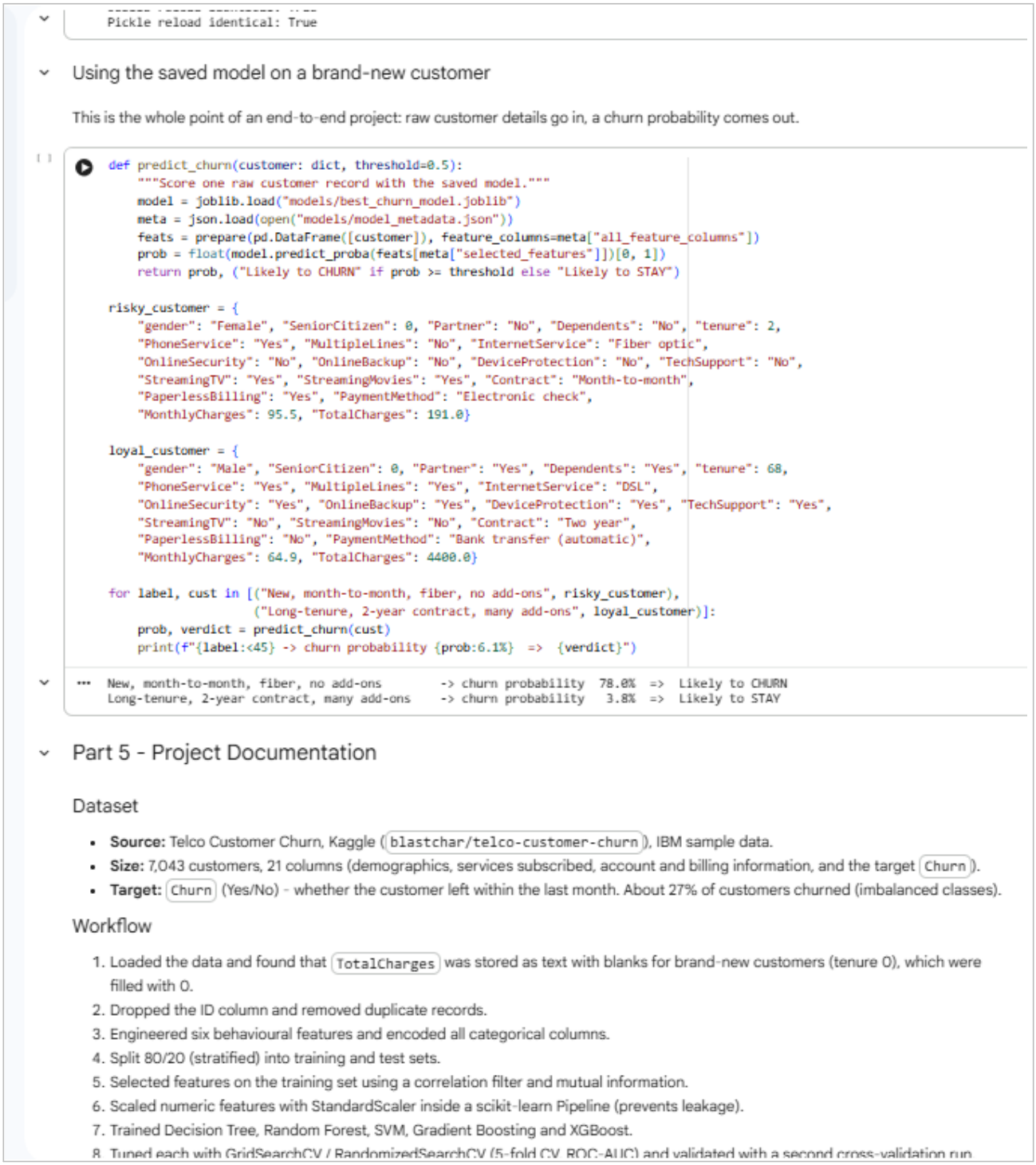
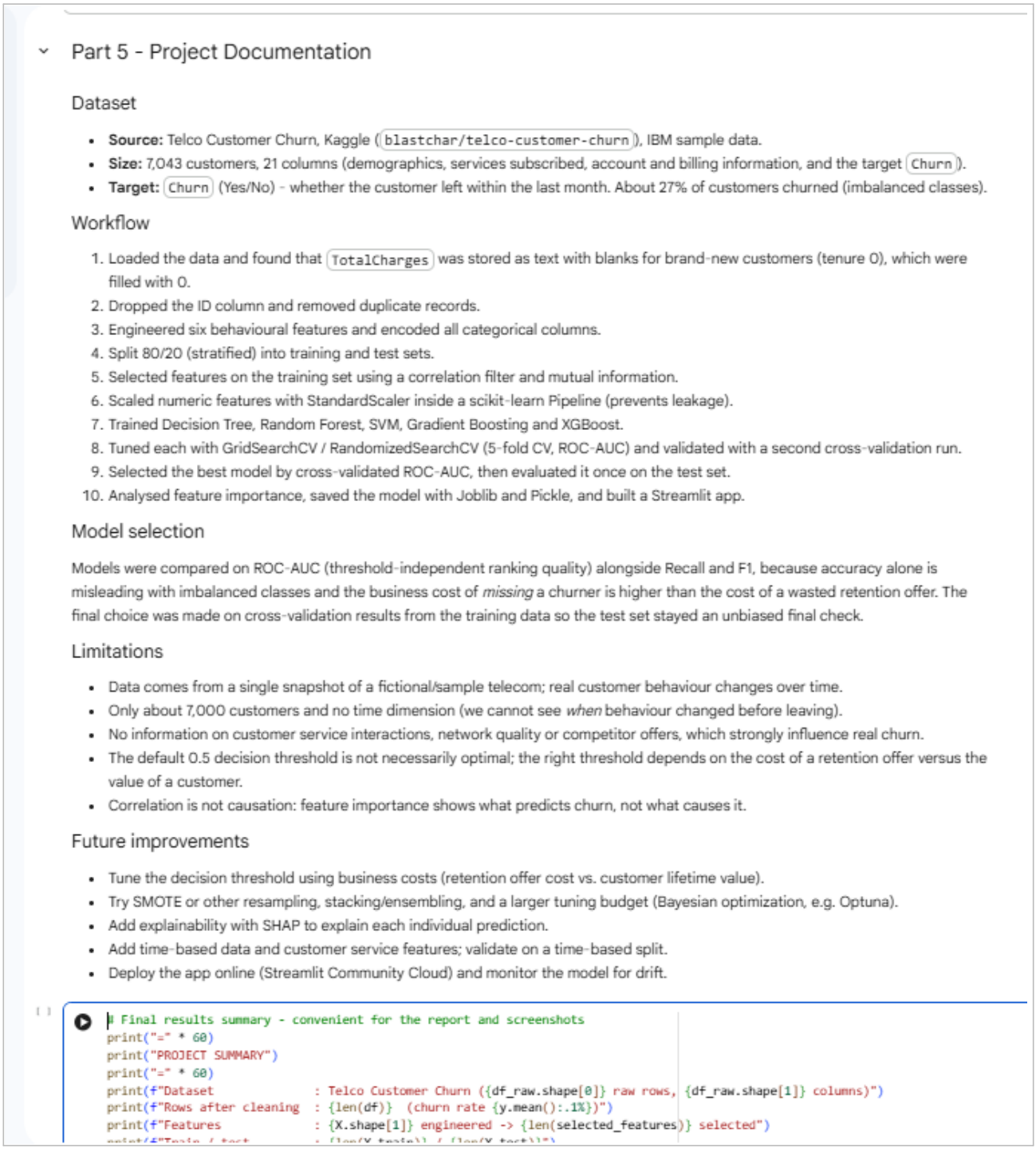


Figure 28: Scoring two brand-new customers with the saved model

The final step of the pipeline is to use the saved model on raw customer data. The function predict_churn() loads the saved model and metadata, passes the customer's details through the same prepare() function used for training, and returns the churn probability. A new customer on a

month-to-month contract with fiber internet and no add-on services received a churn probability of **78.0%** (likely to churn), while a long-tenure customer on a two-year contract with many add-ons received only **3.8%** (likely to stay). These results agree with the patterns found in the data and show that the saved model works end to end.

10. Part 5 - Project Documentation



Part 5 - Project Documentation

Dataset

- **Source:** Telco Customer Churn, Kaggle (`blastchar/telco-customer-churn`), IBM sample data.
- **Size:** 7,043 customers, 21 columns (demographics, services subscribed, account and billing information, and the target `Churn`).
- **Target:** `Churn` (Yes/No) - whether the customer left within the last month. About 27% of customers churned (imbalanced classes).

Workflow

1. Loaded the data and found that `TotalCharges` was stored as text with blanks for brand-new customers (tenure 0), which were filled with 0.
2. Dropped the ID column and removed duplicate records.
3. Engineered six behavioural features and encoded all categorical columns.
4. Split 80/20 (stratified) into training and test sets.
5. Selected features on the training set using a correlation filter and mutual information.
6. Scaled numeric features with StandardScaler inside a scikit-learn Pipeline (prevents leakage).
7. Trained Decision Tree, Random Forest, SVM, Gradient Boosting and XGBoost.
8. Tuned each with GridSearchCV / RandomizedSearchCV (5-fold CV, ROC-AUC) and validated with a second cross-validation run.
9. Selected the best model by cross-validated ROC-AUC, then evaluated it once on the test set.
10. Analysed feature importance, saved the model with Joblib and Pickle, and built a Streamlit app.

Model selection

Models were compared on ROC-AUC (threshold-independent ranking quality) alongside Recall and F1, because accuracy alone is misleading with imbalanced classes and the business cost of *missing* a churner is higher than the cost of a wasted retention offer. The final choice was made on cross-validation results from the training data so the test set stayed an unbiased final check.

Limitations

- Data comes from a single snapshot of a fictional/sample telecom; real customer behaviour changes over time.
- Only about 7,000 customers and no time dimension (we cannot see *when* behaviour changed before leaving).
- No information on customer service interactions, network quality or competitor offers, which strongly influence real churn.
- The default 0.5 decision threshold is not necessarily optimal; the right threshold depends on the cost of a retention offer versus the value of a customer.
- Correlation is not causation: feature importance shows what predicts churn, not what causes it.

Future improvements

- Tune the decision threshold using business costs (retention offer cost vs. customer lifetime value).
- Try SMOTE or other resampling, stacking/ensembling, and a larger tuning budget (Bayesian optimization, e.g. Optuna).
- Add explainability with SHAP to explain each individual prediction.
- Add time-based data and customer service features; validate on a time-based split.
- Deploy the app online (Streamlit Community Cloud) and monitor the model for drift.

```
# Final results summary - convenient for the report and screenshots
print("=" * 60)
print("PROJECT SUMMARY")
print("=" * 60)
print(f"Dataset              : Telco Customer Churn ({df_raw.shape[0]} raw rows, {df_raw.shape[1]} columns)")
print(f"Rows after cleaning  : {len(df)}  (churn rate {y.mean():.1%})")
print(f"Features             : {X.shape[1]} engineered -> {len(selected_features)} selected")
```

Figure 29: Project documentation section of the notebook: dataset, workflow, model selection, limitations and future improvements

- Correlation is not causation: feature importance shows what predicts churn, not what causes it.

Future improvements

- Tune the decision threshold using business costs (retention offer cost vs. customer lifetime value).
- Try SMOTE or other resampling, stacking/ensembling, and a larger tuning budget (Bayesian optimization, e.g. Optuna).
- Add explainability with SHAP to explain each individual prediction.
- Add time-based data and customer service features; validate on a time-based split.
- Deploy the app online (Streamlit Community Cloud) and monitor the model for drift.

```
# Final results summary - convenient for the report and screenshots
print("=" * 60)
print("PROJECT SUMMARY")
print("=" * 60)
print(f"Dataset              : Telco Customer Churn ({df_raw.shape[0]} raw rows, {df_raw.shape[1]} columns)")
print(f"Rows after cleaning  : {len(df)}  (churn rate {y.mean():.1%})")
print(f"Features             : {X.shape[1]} engineered -> {len(selected_features)} selected")
print(f"Train / test         : {len(X_train)} / {len(X_test)}")
print(f"Models trained       : {', '.join(models)}")
print(f"Best model           : {best_name}")
for k, v in final_df.loc[best_name].items():
    print(f"   Test {k:<10}: {v:.4f}")
print("=" * 60)
```

```
============================================================
PROJECT SUMMARY
============================================================
Dataset              : Telco Customer Churn (7043 raw rows, 21 columns)
Rows after cleaning  : 7021  (churn rate 26.4%)
Features             : 35 engineered -> 24 selected
Train / test         : 5616 / 1405
Models trained       : Decision Tree, Random Forest, SVM, Gradient Boosting, XGBoost
Best model           : Gradient Boosting
   Test Accuracy  : 0.7922
   Test Precision : 0.6527
   Test Recall    : 0.4597
   Test F1        : 0.5394
   Test ROC-AUC   : 0.8397
============================================================
```

Figure 30: Final project summary printed by the notebook

The documentation is kept inside the notebook so that it travels with the code. It records the dataset, the ten-step workflow, the reasoning behind the model selection, the limitations and the future improvements. The last cell prints a summary of the project, shown in Figure 30.

10.1 Results

Item	Value
Dataset	Telco Customer Churn, 7,043 raw rows and 21 columns
Rows after cleaning	7,021 (22 duplicates removed), churn rate 26.4%
Features	35 engineered, 24 selected
Train / test split	5,616 / 1,405 customers
Models trained	Decision Tree, Random Forest, SVM, Gradient Boosting, XGBoost
Best model	Gradient Boosting (cross-validated ROC-AUC 0.8474)
Test accuracy / precision	0.7922 / 0.6527
Test recall / F1	0.4597 / 0.5394
Test ROC-AUC	0.8397

Table 4: Summary of final results

10.2 Limitations

- The data is a single snapshot from a sample telecom company, and real customer behaviour changes over time.
- There are only about 7,000 customers and no time dimension, so we cannot see when behaviour changed before a customer left.
- There is no information about customer service interactions, network quality or competitor offers, which strongly influence real churn.
- The default 0.5 decision threshold is not necessarily the best one. The final model catches fewer than half of the churners at this threshold.
- Feature importance shows what predicts churn, not what causes it.

10.3 Future Improvements

- Tune the decision threshold using business costs (the cost of a retention offer compared with the value of a customer).
- Try SMOTE or other resampling, stacking of several models, and a larger tuning budget such as Bayesian optimization.
- Add SHAP explanations so that each individual prediction can be explained.
- Add time-based data and customer service features, and validate the model on a time-based split.
- Deploy the prediction app online and monitor the model for drift.

11. Challenges Faced

- **Hidden missing values.** df.info() reported no missing values, yet TotalCharges was stored as text. Converting it to numbers revealed 11 blanks, all belonging to customers with tenure 0, so they were filled with 0 instead of the mean.
- **Imbalanced classes.** Only 26.4% of customers churn, so a model that always predicts "stay" would score about 73.5% accuracy. Class weights were used and the models were judged on recall, F1 and ROC-AUC instead of accuracy alone.
- **Avoiding data leakage.** The data was split before feature selection, the scaler was placed inside the pipeline so it is fitted only on training folds, duplicates were removed before splitting, and the best model was chosen by cross validation instead of by test score.
- **Long tuning times.** Searching the full grid for the ensemble models would have taken too long, since even 15 random combinations took 70.6 s for Random Forest and 91.2 s for Gradient Boosting. RandomizedSearchCV with a fixed budget gave good results in reasonable time.
- **Precision versus recall.** The model chosen by ROC-AUC (Gradient Boosting) has high precision but low recall, while other models catch many more churners at the cost of more false alarms. Understanding this trade-off was important to interpret the results correctly.
- **Making the pipeline reusable.** Putting all cleaning and feature engineering in one prepare() function was needed so that new customer data is processed exactly like the training data.

12. Conclusion

This project delivered a complete machine learning workflow for predicting customer churn on the Telco Customer Churn dataset: data cleaning, feature engineering and selection, scaling, training of five different models, hyperparameter tuning with cross validation, feature importance analysis, and saving of the final model.

Gradient Boosting was selected as the best model, with a cross-validated ROC-AUC of 0.8474 and a test ROC-AUC of 0.8397 (accuracy 0.7922, precision 0.6527, recall 0.4597, F1 0.5394). Tuning improved every model, and after tuning the five models performed very similarly in ranking customers by risk. The most important drivers of churn were a month-to-month contract, a short tenure and fiber optic internet service. The saved Joblib and Pickle files reload correctly and can score new customers, as shown by the two example predictions.

The main weakness of the final model is its low recall at the default threshold, so the most valuable next steps are to tune the decision threshold for the business cost of missing a churner and to test the model on more recent, time-based data. Overall, the task provided hands-on experience with building a clean, reusable and well-evaluated machine learning pipeline.